

Reviewer Pack — Minimal Rank of Tropicalized Kerdock Codes over DPLL Search ...

Entry 7b65572c6fec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 12:30:50 UTC

Minimal Rank of Tropicalized Kerdock Codes over DPLL Search Tree Width

Entry ID: 7b65572c6fec **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 12:30:50 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Kerdock Codes) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Width

Statement:

[‘For a Kerdock code C with rank r , the minimal rank of its tropicalization T_C is at most 2^r .’, ‘For a CNF formula F with DPLL search tree width t , the minimal rank of the tropicalized Kerdock code constructed from F ’s clause indicators is at least t .’, ‘Thus, $E[\text{rank}(T_C)] = \Theta(2^{(r(C))} + \log(t(F)))$ ’]

Rationale (proposer’s reasoning):

[‘Tropicalization provides a bridge between algebraic structures and their geometric representation. Kerdock codes are known for their interesting properties in the realm of algebraic geometry.’, ‘DPLL search tree width is a natural measure of computational complexity. By relating the tropicalization of Kerdock codes to DPLL search tree width, we may expose new structural insights into complexity classes.’, ‘This conjecture aims to demonstrate a connection between an algebraic-geometric object and a complexity-theoretic measure, potentially leading to new lower bounds or separations.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d0bee5e16a595bd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all seeds, the ratio of the measured minimal rank of T_C to 2^r is within $[0.9, 1.1]$ and greater than or equal to $\log_2(t) - 3$ standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND Kerdock codes AND DPLL search tree width - minimal rank tropicalization Kerdock codes AND complexity theory DPLL - DPLL search tree width application in tropical geometry of Kerdock codes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random Kerdock code with rank r
    n = 30
    r = random.randint(1, 5)
    C = [[random.choice([0, 1]) for _ in range(n)] for _ in range(r)]

    # Compute the tropicalization T_C
    T_C = []
    for i in range(n):
        max_val = -math.inf
        for j in range(r):
            if C[j][i] > max_val:
                max_val = C[j][i]
        T_C.append(max_val)

    # Compute the minimal rank of T_C
    min_rank_T_C = len(set(T_C))

    # Generate a CNF formula F with DPLL search tree width t
    t = random.randint(1, 5)
    F = []
    for _ in range(t):
        clause = [random.choice([f'x{i}', f'~x{i}']) for i in range(n)]
        F.append(clause)
```

```

# Compute the minimal rank of T_C
min_rank_T_C = len(set(T_C))

# Check if the conjecture holds
ratio = min_rank_T_C / (2 ** r)
std_dev = 3 * math.sqrt(ratio * (1 - ratio) / n)
conjecture_holds = ratio >= log_2(t) - std_dev and ratio <= log_2(t) + std_dev

return {
    "metric_name": "min_rank_T_C",
    "metric_value": min_rank_T_C,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Ratio {ratio} not within bounds"
}

def log_2(x):
    return math.log2(x) if x > 0 else float('-inf')

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Ratio out of bounds\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

0.25 not within bounds'
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': 1}

```

```

TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
TRIAL: {'metric_name': 'min_rank_T_C', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds':
RESULT: FALSIFIED counterexample="Ratio out of bounds" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test has only tested a very small number of instances ($n \leq 15$). This is insufficient to confirm the conjecture, as it may not scale with n . Additionally, the metric 'min_rank_T_C' could be saturated or bounded by construction, making any observed bounds trivial.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed, the ratio of the measured minimal rank of T_C to 2^r is not within the bounds [0.9, 1.1], which | next: Further investigation is needed to determine if the observed behavior is due to a flaw in the test setup or if it indicates that the conjecture does not hold for all cases. Additional testing with a larger variety of seeds and under different conditions should be conducted.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14647
2	preregistration	ollama_remote	glm4:latest	0	0	6593
3	novelty	ollama_remote	glm4:latest	0	0	4614
4	novelty	ollama_remote	glm4:latest	0	0	5427
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15397
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17028
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13438
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8868
9	critic	ollama_remote	glm4:latest	0	0	15979
10	judge	ollama_remote	glm4:latest	0	0	6303

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108294 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7b65572c6fec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/7b65572c6fec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7b65572c6fec.tar.gz` (if generated)